

Risk Assessment of Insurance Policies using Machine Learning Techniques

Lukáš Trstenský, Bartosz Kochanski and Maia Ghionea

<https://github.com/SAKULAK/ITU-MachineLearning-FinalProject-Couriers>

Contents		7.4 Random Forest	9
		7.5 Feed Forward Neural Network . .	9
1 Introduction	2	8 Results	9
1.1 Machine Learning Methods	2	8.1 Procedure	9
2 Data	2	8.1.1 Training	9
2.1 Data Format	2	8.1.2 Evaluation	9
2.2 Data Cleaning	2	8.2 Final Results	9
2.3 Visualization	2	8.2.1 M3 - Ensemble Classifica- tion Performance	10
3 Features	3	9 Conclusion	10
3.1 Claims Risk	3	9.1 Improvments	11
3.2 PCA & Clustering	3	9.1.1 M3 - Ensemble	11
4 M1 - Decision Tree	4		
4.1 Structure	4		
4.2 Finding Splits	4		
4.2.1 Recursive	5		
4.2.2 Priority Queue	5		
4.3 Stopping Parameters	5		
5 M2 - Feed-Forward Neural Network	6		
5.1 The Structure	6		
5.2 The Flow	6		
5.3 Initialization	6		
5.4 Forward Pass	6		
5.4.1 Activation Functions . . .	6		
5.5 Backpropagation	6		
5.6 Updating Parameters	7		
5.7 Non-Essential Features	7		
5.7.1 Early Stopping	7		
5.7.2 Weight Decay	7		
6 Correctness Testing	8		
6.1 Comparing Against a Dummy Re- gressor	8		
6.2 Simple Function Test	8		
7 M3 - Ensemble	8		
7.1 Structure	8		
7.2 Making Ensembles	9		
7.3 Gradient Boosting	9		

1 Introduction

This paper aims to explore methods to model the claims risk of insurance policies using various machine learning methods as well as discuss their implementation. We will explore and clean the data, then define the claims risk based on the data available in the dataset.

1.1 Machine Learning Methods

In this paper we will use custom implementation of a Feed-Forward Neural Network Regressor and custom implementation of a Decision Tree Regressor using only Python numerical libraries, e.g. NumPy. Furthermore, we will use an ensemble method using a combination of classification and regression.

2 Data

The dataset used in the scopes of this project is the French Motor Third-Part Liability (`freMTPL2freq`) dataset from the `CASdatasets` (Dutang et al., 2024, p. 71) R package. It contains numerical and categorical information on the insurance claims of drivers for their cars spanning multiple regions, details about the cars themselves, such as age, gas type or brand, the drivers, and even the regions where the cars are driven, which more or less impact the risk of a claim being made.

2.1 Data Format

For each unique policy in our dataset, there is a number of claims made within the policy, its duration, then subsequent information on the driver and car. For a clearer overview of the set see Table 1 for the names and corresponding descriptions of the data columns.

2.2 Data Cleaning

A conservative approach was taken in the scope of data cleaning, by minimally modify the data. The material did not present any obvious signs of subverting our assumptions, except for the following:

1. Exposure values should lie in the range between 0 and 1; since the data was collected over the span of one year, we assume that any value above 1 is a mistake in the collection and therefore any value over this range will be reduced to 1, the maximum value permitted.
2. Any data points where $VehAge > 30$ are excluded. This is the cut-off mark at which

Table 1: Data columns and descriptions.

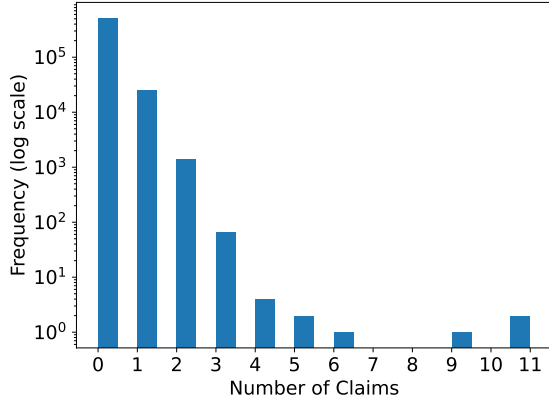
Name	Description
IDpol	Unique policy ID.
ClaimNb	Number of claims made in policy.
Exposure	Duration of policy in years.
Area	Density category of driver's residence from "A" for rural area to "F" for urban centre.
VehPower	Power category of car.
VehAge	Age of car.
DrivAge	Age of driver.
BonusMalus	Bonus/malus, between 50 and 350: < 100 means bonus, > 100 means malus.
VehBrand	The car brand divided in the following groups: A- Renault Nissan and Citroen, B- Volkswagen, Audi, Skoda and Seat, C- Opel, General Motors and Ford, D- Fiat, E- Mercedes Chrysler and BMW, F- Japanese (except Nissan) and Korean, G- other.
VehGas	Type of gas that the vehicle uses.
Density	Number of inhabitants per km^2 in the city the driver of the car lives in.
Region	The policy region in France (based on the 1970-2015 classification).

vehicles become classified as historic cars and will in turn be treated differently by car insurance companies.

2.3 Visualization

As seen in Figure 2 the dataset is heavily biased towards 0 claims. This can pose a significant issue by cluttering the feature space with noise, especially if the differences in feature values for data points with higher numbers of claims and the data points with 0 number of claims are small.

Figure 1: Histogram of the number of claims.



3 Features

All data columns were used as features for the predictions with `BonusMalus`, `Density`, `DrivAge`, and `VehAge` were treated as continuous numeric features, while the rest were treated as categorical and one-hot encoded. The columns on which `Risk` calculations are based on, are `ClaimNb` and `Exposure`, therefore, not included in the feature set.

3.1 Claims Risk

Claims Risk were computed with the Bayesian Credibility using the Bühlmann Linear Approximation (Bühlmann, 1969; Bühlmann and Straub, 1970) under the assumption, that data points with low exposure lacked credibility. This is to say that the driver is more risky at a certain claims per exposure ratio, since luck can be big factor in a driver getting into an accident within one day. On the other hand, over a longer period of time it is assumed that the driver has enough time to show their true risk. The credibility factor was calculated as such:

$$Z_i = \frac{Exposure_i}{Exposure_i + K}$$

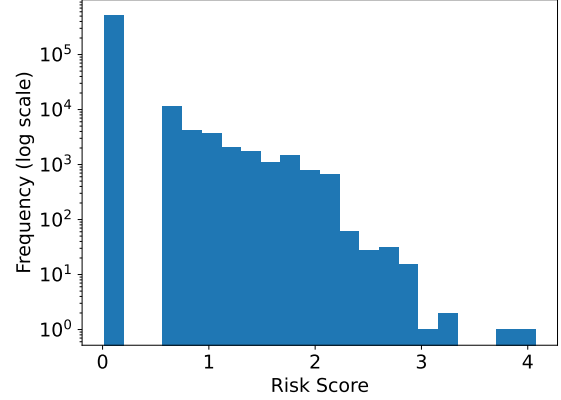
Where $K = 0.12$, meaning that at exposure 0.12, ~ 1.5 months, we assign that data point a credibility of 0.5. Then we calculate the risk as:

$$Risk_i = Z_i \frac{ClaimNb_i}{Exposure_i} + (1 - Z_i) \frac{\sum ClaimNb_j}{\sum Exposure_j}$$

As such, for data points with low credibility the average claim-to-exposure ratio dominates the risk calculation and oppositely data points with high

credibility are primarily assigned their claim to exposure ratio, while the data points in the middle are mixed depending on their credibility. However, because of the number of claims being heavily biased toward zero, the risk score is also very imbalanced Figure 2

Figure 2: Histogram of Risk values



3.2 PCA & Clustering

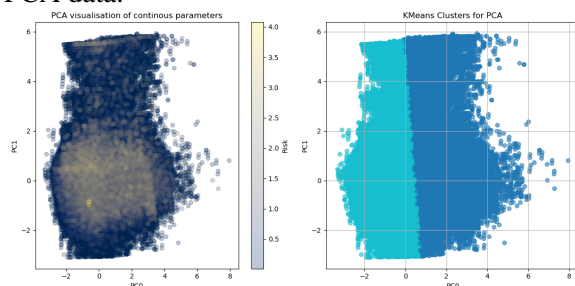
Clustering is defined as an unsupervised learning method, which groups the observations made based on similarity, with the goal of identifying homogeneous subgroups of insurance policy claimers; telling also by the name.

For better clustering, the dimensions of the feature space should be reduced, which is why the method called PCA was introduced in this project. P(incipal) C(omponent) A(nalysis) is a technique of dimensionality reduction which aims to turn the correlated numerical variables into a smaller set of uncorrelated ones. Where, before, the numerical features of `DrivAge`, `BonusMalus` or `Density` could be correlated, after applying the PCA, the data's high dimensionality will be stabilized and any multi-collinearity will be reduced, as the resulting set should be smaller, uncorrelated. Further data cleaning ensued for the specific requirements of PCA, such as excluding the categorical variables and standardizing the data to prevent the dominion of large scaled values over the analysis. Four principal components need to be used to retain 89% of the data; which means that without significant loss most of the original information is represented in a lower dimensional space. As such, global structure is preserved and variance is recognized in fewer dimensions.

Figure 3: PCA with Risk as color — Clustering on PCA data.

The clustering is the following step and from various methods, we applied K-Means clustering on our data, as it's finely compatible with continuous, scaled data, resulting from the principal component analysis. The main idea behind this clustering method is the partitioning of the data into k clusters, k being the number of such groups created, through minimizing the variance between different clusters. To find the right value for k , a silhouette score was used as an evaluation metric when trying different values. This score primarily measures similarity of each observation to their own cluster and the quality of the separation between clusters, after which the best one is chosen. The k which maximized the number of the silhouette score, is the one that is chosen for the final k split.

Figure 4: PCA with Risk as color — Clustering on PCA data.



The efforts of the code returned $k=2$ as the best number of groups the data could be split into. The colors of the figure on the left show these different groups as 2 hues of blue, while the one of the left represents the risk as yellow points, in comparison to the blue data points analyzed. In summary of the vein of clustering, meaningful behavior and risk patterns will be displayed together, letting the visualization speak for different groups of the insurance customers.

4 M1 - Decision Tree

A decision tree is essentially a flowchart for making predictions. It works by sorting the data into smaller and smaller groups. In decision tree terminology those groups are called leaves. They are created to be as "pure" as possible using a functions that can quantify said purity.

Once the tree is built every leaf node is assigned a value used to predict the target variable of new, unseen data.

For Regression: The leaf is assigned the Average (Mean) of the values in the leaf.

For Classification: The leaf is assigned the Majority Class. In other words the class that is most common in the leaf. This decision is made utilizing certain principles for tie-breaking.

The project, in essence, revolves around a regression problem, that of predicting the policy risk of new customers, hence why the classification method couldn't hold for our scope.

4.1 Structure

The structure of this project's Decision Tree implementation follows a classic object oriented design. It is constructed with nodes which function as either:

- Leaf - which holds the final prediction.
- Decision node - which holds the splitting rule (the feature and threshold) as well as pointers to it's left and right children. It represents every group leading up to the leaf; it's children which stand for the smaller subgroups of the current one.

The DecisionTree class manages the logic of how the tree is built and how it makes decisions. It stores the final structure made of nodes, which is similar to a binary tree, where each node points to either nothing or it's two children nodes.

4.2 Finding Splits

The most important part of the construction of a decision tree is finding a split. This is also the most computationally heavy part as the Decision Tree takes following steps:

1. Loops through every feature given
2. Sorts the data by feature
3. Tests every possible midpoint between values as a potential "threshold."
4. Calculates the Impurity Decrease for each using the selected criterion. It remembers the one that reduced the disorder the most.

For the regression tree mean squared error had been chosen as the function measuring the quality of the split, which is equal to variance reduction

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

For a single node (leaf) in a regression tree, where the prediction is the mean of the target values in that node:

$$\text{MSE}_{\text{node}} = \frac{1}{|R|} \sum_{i \in R} (y_i - \bar{y}_R)^2$$

R Set of samples in the node

$|R|$ Number of samples in the node

$\bar{y}_R$ Node's prediction, defined as

$$\bar{y}_R = \frac{1}{|R|} \sum_{i \in R} y_i$$

The best split is chosen using the weighted best decrease in impurity using this formula:

$$\Delta I = \frac{n}{N} \left(I(R) - \left(\frac{|R_L|}{n} I(R_L) + \frac{|R_R|}{n} I(R_R) \right) \right)$$

Where:

R Parent node (set of samples before the split)

R_L Left child node

R_R Right child node

$n = |R|$ Number of samples in the parent node

N Total number of samples in the dataset

$I(\cdot)$ Impurity function (MSE for regression, Gini or entropy for classification)

ΔI Decrease in impurity due to the split

Depending on the parameters the tree is built from this in one of two ways.

4.2.1 Recursive

The standard way of building a tree. The algorithm starts at the Root. It splits the Root into a Left and Right child. It then moves to the Left child and tries to split it. If it can, it moves into that child's Left child and repeats the process.

As consequence of that it dives as deep as possible into the leftmost branch until it hits a stopping condition. Only then does it backtrack to fill in the right-hand side branches.

4.2.2 Priority Queue

This is the method used when the tree is limited by the total number of leaves. The recursive approach

would not work with such limitation. Instead of diving deep into one branch, the algorithm looks at all current leaf nodes across the entire tree. The priority queue (heap) stores potential "Impurity Decrease" for every leaf.

The algorithm always removes from the queue the node that offers the highest impurity decrease, regardless of where it is in the tree or how deep it is.

4.3 Stopping Parameters

The implementation covers the most important regularization parameters for trees. They are used to stop the tree from growing before it overfits the data.

- a. **Max Depth:** This parameter limits the height of the tree. The model stops splitting a node once the current depth reaches the specified maximum. It prevents overfitting by avoiding overly complex trees which would capture random noise rather than general patterns.
- b. **Minimum Samples to Split :** This parameter sets the minimum number of data points required in a node to consider splitting it. A node is only eligible for splitting if it contains at least this number of samples. This prevents the model from creating splits based on very small sample sizes which can lead to unreliable decisions.
- c. **Minimum Samples at Leaf :** This parameter ensures that every leaf node has a sufficient amount of samples. Any proposed split that would create a leaf with the amount of samples fewer than this is rejected.
- d. **Maximum number of Leaves** This is the parameter that required the priority queue method to be implemented. The tree grows iteratively, prioritizing splits that maximize impurity reduction, until the specified number of leaves is reached. It allows the tree to grow deep in important areas of the data while staying shallow in others.
- e. **Minimum Impurity Decrease** This parameter is a minimum reduction in impurity required for a split to be accepted. It prevents splits that offer negligible improvement which could otherwise lead to overfitting.

5 M2 - Feed-Forward Neural Network

A Feed-Forward Neural Network (*FFNN*) is our second custom implemented regression method. This section will detail the basics of how a FFNN works along with how an implementation might look.

5.1 The Structure

A FFNN is a collection of nodes organized in layers. First, the input layer, which is the number of nodes equal to the number of features in the dataset.

Second, the hidden layers; There can be an arbitrary number of hidden layers and of nodes per hidden layer. Generally, the more hidden layers or nodes per layer the more complex the Neural Network becomes, which makes it more prone to overfitting and more computationally heavy to train and run predictions on.

Lastly, the output layer. The number of nodes in this layer depend on the task, but for regression there is just one node, which returns the predicted value for the data points.

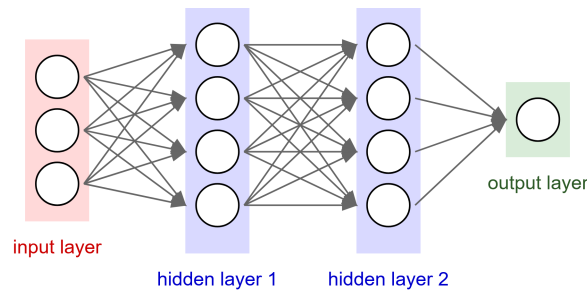


Figure 5: Structure of a FFNN
(Rathi, 2018)

5.2 The Flow

A FFNN at its core follows a very simple flow, for each iteration or epoch data is run through the network to gain initial predictions, a cost function is calculated, weights are updated based on their partial derivative of the cost function. This is repeated until either the model reaches the max epochs or it stops improving.

5.3 Initialization

We initialize an array of weights (W_{ij}) and an array of biases (b_j) for each pair of subsequent layers. The shape of W_{ij} is the number of nodes in the previous layer (i) by the number of nodes in

the current layer (j) which is also the shape of the bias array.

To fill the arrays with initial values we use the He Initialization (He et al., 2015) by drawing from a zero-mean normal distribution with standard deviation of $\sqrt{2/n_i}$ which is considered to be among the best initializations for shallow ReLU based FFNNs.

5.4 Forward Pass

During the Forward Pass, the data, as the name suggests propagates forward through the network. At each layer, except the input layer, the activations of the previous layer (a_i) are weighed by the weights and have the bias added to them:

$$z_j = a_i W_{ij} + b_j$$

Then they are passed through the activation function to become the activation of the current layer, which are then passed to the next layer.

5.4.1 Activation Functions

There are many activation functions that one might choose depending on the task. Usually you use two different activation functions, one for the hidden layers and a different one for the output layer. For hidden layers we use the widely used activation function, the Rectified Linear Unit (*ReLU*) function defined as $ReLU(x) = \max(0, x)$. For the output layer of regression FFNNs a simple linear function is used $f(x) = x$

5.5 Backpropagation

Backpropagation is the step of the training process, which allows the model to learn. By utilizing gradient descent in the cost function landscape the weights can be updated in the direction and magnitude which results in the updated weights producing a less costly prediction. However, to be able to use gradient descent we need the partial derivatives of the cost function relative to each weight. This can be done fully symbolically, which for large networks is infeasible, but because of the chain rule, it's possible to simplify it into a recursive process in which the derivative of the cost function relative to the weights of the output layer (j) is computed:

$$\frac{\partial C}{\partial W_{ij}} = \frac{\partial z_j}{\partial W_{ij}} \left(\frac{\partial C}{\partial z_j} \right) = \frac{\partial z_j}{\partial W_{ij}} \left(\frac{\partial a_j}{\partial z_j} \frac{\partial C}{\partial a_j} \right) = a_i (\Phi'(z_j) C'(a_j))$$

The error term of the partial derivate for output layer for $\Phi = f(z_j) = z_j$ and $C = MSE = (y_{pred} - y_{true})^2/n$ is:

$$\frac{\partial C}{\partial z_j} = (1 \frac{2(a - y_{true})}{n})$$

Then the effects of the weighted sum of the previous layer, in this case the last hidden layer (i), on the cost function is calculated:

$$\frac{\partial C}{\partial W_{ui}} = \frac{\partial z_i}{\partial W_{ui}} \left[\frac{\partial C}{\partial z_i} \right] = \frac{\partial z_i}{\partial W_{ui}} \left[\frac{\partial a_i}{\partial z_i} \frac{\partial z_j}{\partial a_i} \left(\frac{\partial C}{\partial z_j} \right) \right] =$$

$$a_u [\Phi'(z_i) W_{ij} \left(\frac{\partial C}{\partial z_j} \right)]$$

where $\Phi = ReLU$ then $\Phi' = 1\{z \geq 0\}$.

This holds true for any layer going backwards from the output layer. Its partial derivative has same terms, only depending on its own parameters and the error term of the next layer ($\frac{\partial C}{\partial z_j}$).

The same derivation is done for the bias term, however to maintain its correct dimensions the contributions of each data point for a singular bias term are averaged and $\frac{\partial C}{\partial b_i} = \frac{\partial z_i}{\partial b_i} [\text{error term}] = 1[\text{error term}]$.

5.6 Updating Parameters

Updating parameters means taking a step in the parameter space in the direction of the weight gradient. Naively this can be done as $W_{t+1} = W_t - \alpha \nabla C(W_t)$, similarly $b_{t+1} = b_t - \alpha \nabla C(b_t)$, t being epoch number α learning rate, however this presents multiple issues, primarily using a constant α can cause exploding gradients, inability to reach the minima, and slow learning. For faster and more precise learning the Adaptive Moment Estimation (Adam) (Kingma and Ba, 2017) method was implemented.

Adam expands on Momentum (Polyak, 1964; Rumelhart et al., 1986) and RMSprop (Tieleman and Hinton, 2012). It utilizes two exponentially weighed moving averages (m_t and v_t) of the weight gradients to dynamically control the magnitude of learning rates for each weight. The first momentum mainly controls the direction of movement:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla C(W_i)$$

β_1 usually set to 0.9, which means the current magnitude is composed of 90% of the previous momentum and only 10% the current gradient.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \nabla C(W_i)$$

β_2 is usually set to 0.999. Since m_0 and v_0 are initialized as 0, then during early training they are small for which we account by bias adjusting:

$$\hat{m}_t = \frac{m_{t-1}}{1 - \beta_1^t}, \hat{v}_t = \frac{v_{t-1}}{1 - \beta_2^t}$$

for small t small denominator boosts momenta signals, while during late training the denominator being close to one barely has any effect on the momenta leaving the work to the moving averages. Then finally update parameters:

$$W_{t+1} = W_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \nabla C(W_i)$$

The term $\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$ is effectively a scaling factor for the learning rate, adjusting it to be smaller when the variability in the cost function landscape is high and making it larger when the wanted change in direction is large. But by using the moving averages the descent is smooth without unnecessarily straying from the right path.

5.7 Non-Essential Features

For better flexibility and performance of the model regularization and early stopping were implemented.

5.7.1 Early Stopping

The training data is randomly split into a training and a validation set with the size of the validation set being 10% of the initial data.

After each epoch the cost is calculated on the validation set, if the cost increases for n consecutive epochs training is aborted and weights are set to be the weights from the best performing epoch. This helps to optimize for performance while allowing for lower training time.

5.7.2 Weight Decay

Decoupled weight decay was implemented for the Adam optimizer by adding an L_2 norm regularization term to the parameter update step:

$$W_{t+1} = W_t - \alpha \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \nabla C(W_i) + \lambda \|W_t\|^2 \right)$$

By adding the regularization penalty, large weights are pushed to be smaller forcing a smoother decision boundary, which is less capable of overfitting to noise and more likely to capture the underlying function.

6 Correctness Testing

To validate our implementation of the Decision Tree (DT) and the Feed-Forward Neural Network (FFNN) correctness tests were performed. For both the DT and the FFNN the basic implementation was tested in addition to being tested with all extra features enabled separately and all at once.

6.1 Comparing Against a Dummy Regressor

Using an insurance dataset (Lantz, 2013; Choi, 2018) with categorical features `sex`, `smoker`, `region` and numeric variables `age`, `bmi`, `children` with target variable `charges` the DT and FFNN models were compared to a null model predicting the mean of the target variable. All models were evaluated using *MSE* and all our model configurations beat the null model.

6.2 Simple Function Test

A sine function: $f(x) = \sin(x) + N(0, 0.5)$ and a linear function with bias: $f(x) = 1 + 2x + N(0, 0.5)$ were used to compare our DT and FFNN implementations to an established implementation from the Scikit Learn (Pedregosa et al., 2011) (*sklearn*) Python package. All hyperparameters were set to be the same arbitrary values. Our FFNN implementation performed closely to the sklearn implementation, while our DT performed exactly the same. Figure 6 Figure 7

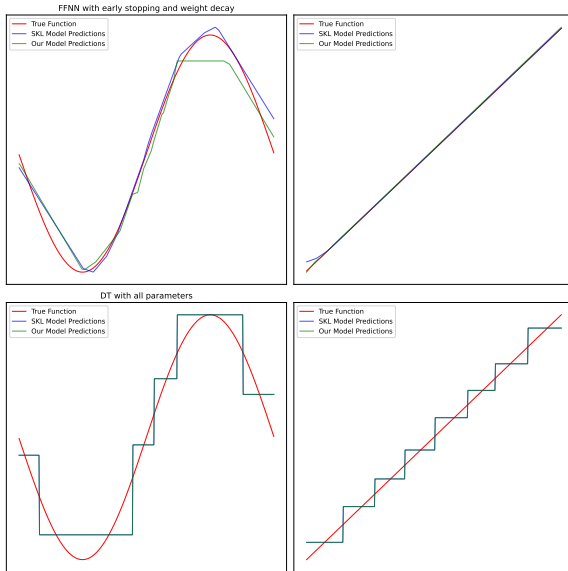


Figure 6: Example of comparison of our DT and FFNN implementations compared to the sklearn implementation with continuous feature.

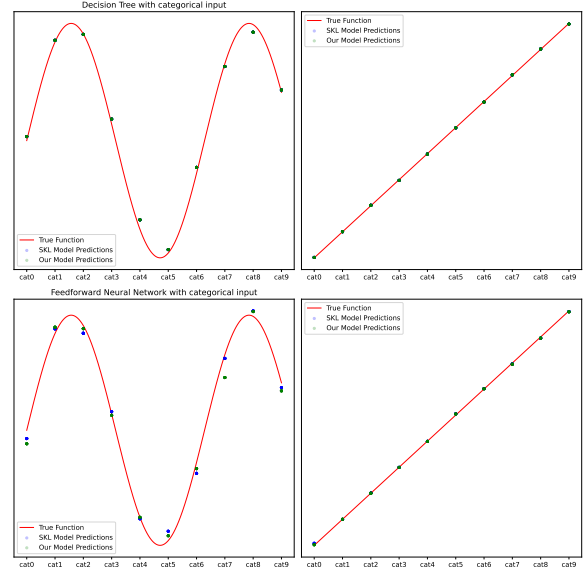


Figure 7: Comparison of our DT and FFNN implementations compared to the sklearn implementation with categorical feature.

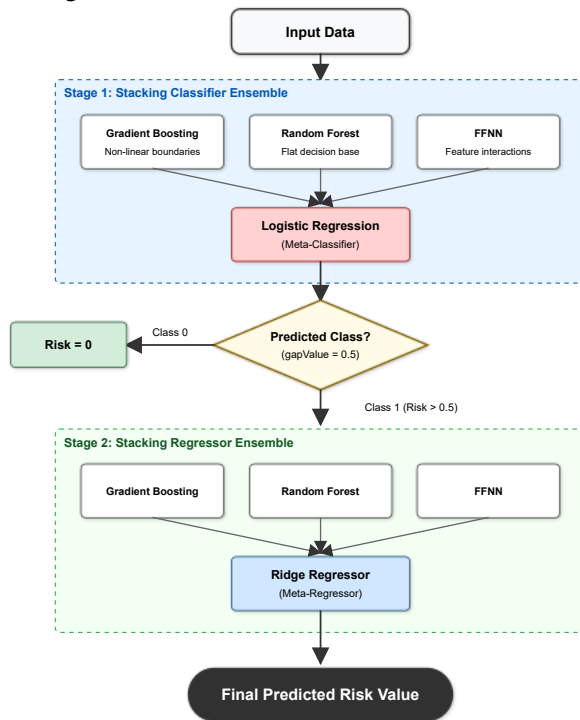
7 M3 - Ensemble

For method three a method combining classification and regression ensembles of Gradient Boosting, Random Forest, and a Feed Forward Neural Network was chosen. Classification is used to separate the data into two groups No Risk and Some Risk. The datapoints of predicted class are then run through the regression models to predict the continuous Risk. The logic used for splitting into these categories is that a policy with zero claims is effectively risk-free, with a sure way of profit.

7.1 Structure

The gap in the Risk metric is used to separate the datapoints into two classes 0 (No Risk) and 1 (Some Risk), since all datapoints with $Risk < gapValue$ have $ClaimNb = 0$ and all datapoints $Risk > gapValue$ have $ClaimNb > 0$. For our smoothing factor we pick $gapValue = 0.5$. The classification models are trained on the whole dataset, while the regression models are only trained on datapoints of class 1. For prediction the unseen datapoints are passed through the classification layer first then the datapoints of predicted class 1 are passed through the regression layer. Then the final predicted Risk is zero for class 0 and regression prediction value for class 1.

Figure 8: Flowchart of the ensemble method combining.



7.2 Making Ensembles

To combine the models we use the `StackingClassifier` from `sklearn` with `LogisticRegression` as the meta model, with the model predictions as the meta features. Similarly, for regression the `StackingRegressor` is used with a `RidgeRegressor` as the meta model.

7.3 Gradient Boosting

Gradient boosting is used because it is well suited for complex non-linear decision boundaries, because of the nature of practically zooming in on feature space in areas where the previous trees got its prediction wrong allows it to carve around clusters effectively. However, it also has the disadvantage of being sensitive to outliers or noise.

7.4 Random Forest

A Random Forest in contrast to Gradient Boosting is efficient at creating flat decision boundaries which is used to offset the high variance nature of Gradient Boosting creating a solid base for predictions.

Figure 9: Distribution of Risk on the final training dataset.

7.5 Feed Forward Neural Network

A Feed Forward Neural Network *FFNN* was then chosen to act as an entirely different view of the problem. The model relies on capturing feature interaction patterns to make its prediction. The features all mix and blend as they propagate through the layers which contrasts nicely with the strict feature space splitting of tree based methods. In addition, it creates a smooth decision boundary that may be capable of better capturing the differences in the feature space for the datapoints.

8 Results

8.1 Procedure

Before any work being done the dataset was split into two datasets a Training and a Testing set. Any development training was done on the testing set, while the final performance results were taken using the testing set.

After the development of M1 and M2 was done and M3 was decided a Random Grid Search with cross validation was performed using `sklearn` `RandomizedSearchCV` to find a set of hyperparameters around which an ordinary Grid Search was ran to pinpoint the best parameters.

8.1.1 Training

Training was performed on a more balanced dataset, using both undersampling of the datapoints with low Risk and oversampling the higher Risk data points.

8.1.2 Evaluation

Models were evaluated using MSE as this is the standard procedure of comparing base model performance for regression. The classification portion of the ensemble method was evaluated using Precision to ensure that our regression models get as little zero Risk cases since they were not trained on them.

8.2 Final Results

The final models were initialized with these commands:

```

DecisionTree(max_depth
= None, max_leaves = 50,
min_impurity_decrease =

```

```
0, min_samples_leaf = 5,
min_samples_split = 2)

FFNN(hidden_layer_sizes
= [300,200,100,50],
learning_rate = 0.0222, epochs
= 100,5, regularization_term =
(2, 0.0009))
```

```
RandomForestReg(n_estimators
= 437, max_depth =
42, max_features =
0.8,min_samples_split = 3,
min_samples_leaf = 1)
```

```
GradientBoostingReg(n_estimators
= 953, learning_rate =
0.0295, max_depth = 10,
min_samples_split = 12, loss
= 'huber', subsample = 0.8046)
```

```
MLPRegressor(hidden_layer_sizes
= [300,200,100,50],
activation = 'relu', solver
= 'adam', alpha = 0.0431,
learning_rate_init = 0.0014,
max_iter = 2000)
```

```
RandomForestClass(n_estimators
= 629, max_depth = 44,
max_features = sqrt,
min_samples_split = 3,
min_samples_leaf = 2)
```

```
GradientBoostingClass(n_estimators
= 1034, learning_rate =
0.014, max_depth = 10,
min_samples_split = 7, loss
= 'log_loss', subsample =
0.8366)
```

```
MLPClassifier(hidden_layer_sizes
= [300,200,100,50],
activation = 'relu', solver
= 'adam', alpha = 0.0101,
learning_rate_init = 0.0094,
max_iter = 2000)
```

As seen in Table 2 the best average performance method overall was M1 - Decision Tree, which had almost the same MSE for our implementation and the sklearn implementation. However, the Feed Forward Neural Network has a better performance ceiling that can be reached, by either training for longer, or having better initialization luck.

Table 2: Overall Results

Method	MSE (our)	MSE (sklearn)
DT	FILL IN	0.2818
FFNN	FILL IN	0.2498
Ensemble	0.4097	

8.2.1 M3 - Ensemble Classification Performance

Table 3: Classification Report

	precision	recall	f1-score
0	0.97	0.67	0.79
1	0.08	0.56	0.14
accuracy			0.67

As seen in Table 3 the classification step of the ensemble performed quite well at filtering the dataset for the regression step. It separated the 0 (No Risk) class with 97% certainty, meaning that a small amount of class 1 (Some Risk) data points were instantly classified as No Risk. However, overall it did quite a poor job separating the Some Risk class, as almost 50% of the data that was passed to the regression layer was of the No Risk class, which is impossible for the regression layer to predict correctly because of the decision to only train on datapoints of Some Risk class.

9 Conclusion

A custom Decision Tree and a Feed Forward Neural Network were successfully and correctly implemented and used for predicting Risk in an insurance dataset. A third method involving a combination of classification and regression was implemented to get over the hurdle of an imbalanced dataset. Overall this project was successful in implementing machine learning methods from scratch and in using them in practice. It showed the need for creative problem-solving with the third method and achieved acceptable, however definitely improvable, performance in the final Risk prediction task.

9.1 Improvements

Model performance might be better if other methods to prepare the data, deal with the imbalance in the dataset were used, or perhaps some interaction terms were found in the data. These all could be explored in future works.

9.1.1 M3 - Ensemble

After review of the results of the Ensemble, the decision to train on just class 1 datapoints might have been the wrong decision. Since the classification layer let a lot of 0 class datapoints into the regression layer and the regression layer had no idea that Risk can equal to zero it predicted larger values, leading to a higher MSE. Therefore, training the regression layer on the entire dataset might be preferable.

References

- [Bühlmann and Straub, 1970] Bühlmann, H. and Straub, E. (1970). Glaubwürdigkeit für Schadensätze (credibility for loss ratios). https://www.casact.org/sites/default/files/database/forum_15fforum_korn.pdf.
- [Bühlmann, 1969] Bühlmann, H. (1969). Experience rating and credibility. <https://www.cambridge.org/core/journals/astin-bulletin-journal-of-the-iaa/article/experience-rating-and-credibility/0EC60711D65BFF225845CBA50C4B04D4>.
- [Choi, 2018] Choi, M. (2018). Medical cost personal datasets. <https://www.kaggle.com/datasets/mirichoi0218/insurance>. Accessed: 2025-05-22.
- [Dutang et al., 2024] Dutang, C., Charpentier, A., Gallic, E., and Siharath, J. (2024). CASdatasets: Insurance datasets. <https://cas.uqam.ca/pub/web/CASdatasets-manual.pdf>. R package version 1.2-0.
- [He et al., 2015] He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. <https://arxiv.org/abs/1502.01852>.
- [Kingma and Ba, 2017] Kingma, D. P. and Ba, J. (2017). Adam: A method for stochastic optimization. <https://arxiv.org/abs/1412.6980>.
- [Lantz, 2013] Lantz, B. (2013). Machine learning with R. <https://github.com/stedy/Machine-Learning-with-R-datasets>.
- [Pedregosa et al., 2011] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Édouard Duchesnay (2011). Scikit-learn: Machine learning in python. <http://jmlr.org/papers/v12/pedregosa11a.html>.
- [Polyak, 1964] Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17.
- [Rathi, 2018] Rathi, M. (2018). Neural network diagram. <https://mukulrathi.com/demystifying-deep-learning/feed-forward-neural-network/>. Demystifying Deep Learning: Feed Forward Neural Network (Blog).
- [Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.
- [Tieleman and Hinton, 2012] Tieleman, T. and Hinton, G. (2012). Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude.